

Enhancing Bug Report Templates in the TianoCore UEFI Firmware Development Community

Laura Baird
lbaird@uccs.edu

Department of Computer Science,
University of Colorado Colorado
Springs
Colorado, USA

Neelesh Reddybattula
nreddyba@uccs.edu

Department of Computer Science,
University of Colorado Colorado
Springs
Colorado, USA

Nazanin Siavash
nsiavash@uccs.edu

Department of Computer Science,
University of Colorado Colorado
Springs
Colorado, USA

Terrance E. Boulton
tboulton@uccs.edu

Department of Computer Science,
University of Colorado Colorado
Springs
Colorado, USA

Armin Moin
moin@purdue.edu

School of Applied and Creative
Computing, Purdue University
Indiana, USA

Abstract

We propose enhancing the bug report templates in the GitHub Issues issue tracking system used by the TianoCore open-source community with the aim of improving the bug triage and resolution process. We analyze the bug repository data and find patterns of information that are useful for bug triage and fixing. However, some of them are only occasionally included in the free-form text of bug reports. Therefore, we propose adding a few new fields to the existing TianoCore bug report template. In this study, we focus on the key TianoCore project, EDK II, which constitutes the core of the UEFI firmware across various firmware vendors and original equipment manufacturers. This study is currently a work-in-progress. So far, we have interviewed a few developers to obtain their feedback and adjust the proposed approach. We are planning more interviews with the TianoCore community to conduct A/B tests and validate our approach to achieve effective and efficient bug triage and resolution.

CCS Concepts

- **Software and its engineering** → **Software maintenance tools**;
- **Security and privacy** → *Software and application security*.

Keywords

firmware, uefi, tianocore, bug triage, bug report, software maintenance

ACM Reference Format:

Laura Baird, Neelesh Reddybattula, Nazanin Siavash, Terrance E. Boulton, and Armin Moin. 2026. Enhancing Bug Report Templates in the TianoCore

UEFI Firmware Development Community. In *Proceedings of the 1st International Workshop on Firmware Testing and Analysis (FTA '26)*, October 04–09, 2026, Oakland, CA, USA. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3842651.3843181>

1 Introduction

Prior work has shown that developers value detailed reproduction steps, stack traces, test cases, and other structured evidence in bug reports in issue tracking systems. However, issue reporters often omit or bury this information in free-form text [3, 4, 12, 18]. This mismatch is especially costly in firmware projects, where a report may depend on pre-operating-system execution state, firmware build options, CPU architecture, boot path, virtualized versus physical hardware, security-sensitive update, and runtime-service behavior. In such settings, the same symptom can imply different triage paths depending on whether it is a regression, whether it is architecture-specific, and whether the affected package is shared by downstream firmware stacks.

The TianoCore EDK II is an open-source reference implementation of the Unified Extensible Firmware Interface (UEFI). It is used across many firmware vendor stacks, which makes it a crucial component of the broader UEFI firmware supply chain. We focus on improving security and software maintenance workflows in this ecosystem. We make bug reports better structured to support bug triage and resolution. The EDK II project has migrated from Bugzilla to GitHub Issues. Its current bug report template has various structured fields, such as impacted packages, build targets, urgency, and maintainer feedback requests [14, 15]. These fields are useful. However, our manual review and analysis of issues suggests that two firmware-specific signals remain under-structured: i) Bug report category, and ii) Affected CPU architectures. Both items matter for bug triage and resolution.

The contribution of this paper is threefold. First, it proposes to add new, pre-defined fields to bug reports in the EDK II issue tracking system to guide bug reporters in creating more complete issues. This way, we expect an increase in the effectiveness and efficiency of the work of bug triagers and bug fixers. Second, it proposes employing Artificial Intelligence (AI), in particular, Generative Pre-trained



This work is licensed under a Creative Commons Attribution 4.0 International License. FTA '26, Oakland, CA, USA

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2969-0/2026/10
<https://doi.org/10.1145/3842651.3843181>

Transformer (GPT) Large Language Models (LLMs), to assist human triagers in their work by showing them some generated advisory comments. Last but not least, it presents a web-based prototype that enables large-scale validation and evaluation using feedback from the TianoCore developers community.

The remainder of this paper is structured as follows. Section 2 reviews related work. Furthermore, Section 3 proposes the novel approach. We also present some preliminary validation results in Section 4. Finally, Section 5 concludes the paper and suggests future work.

2 Related Work

Zimmermann et al. [18] showed that developers and bug reporters often disagree about what information is most useful and easiest to provide in bug reports. Soltani et al. [12] further studied the significance of bug report elements and found that elements such as reproduction steps and stack traces affect debugging and resolution outcomes. Chaparro et al. [3, 4] developed techniques for detecting missing information in bug descriptions and assessing the quality of steps to reproduce. These studies have motivated our approach's premise that bug reporting interfaces should ask explicitly for information developers need rather than relying on reporters to determine what should be included.

GitHub issue templates and forms have become common mechanisms for improving report quality. Li et al. [7] studied bug report templates on GitHub and found that templates can improve report comprehensibility and consistency. In a separate study, Li et al. [8] (other authors despite the name similarity) studied issue and pull-request templates more broadly, showing that template use has nuanced effects on issue discussion and resolution processes. Moreover, Zhang et al. [17] analyzed bug report templates across open-source software projects and derived design guidelines for maintaining them. Nikeghbal et al. [9] explored automated generation of issue report templates. Song et al. [13] proposed interactive bug reporting through a chatbot that guides end users and verifies report quality. Our proposed approach differs from these general-purpose studies by focusing on firmware-specific triage fields and the TianoCore software supply chain context.

We assist in bug triage by providing human triagers with useful information. The bug triage process typically includes invalid and duplicate detection, assignee recommendation, and severity prediction. There exist prior work in the area of Mining Software Repositories (MSR) concerning semi-automated and/or automated bug triage. Qian et al. [10] surveyed bug de-duplication methods. He et al. [6] proposed a convolutional neural network approach to duplicate bug report detection. Choetkiertikul et al. [5] recommended components for issue reports using deep learning. Some other recent LLM-based work, for example, by Acharya and Ginde [1] investigated generating or improving structured bug reports.

3 Proposed Approach

Based on repository analysis and developer feedback, we enable a number of domain-specific diagnostic fields in the TianoCore bug report structure. For instance, as depicted in Figure 1, we add a new field for the impacted architectures, with pre-defined options (multiple items can be selected). The options are the supported

CPU architectures in TianoCore for the UEFI firmware. Here are the current options: IA32, X64, AARCH64, LOONGARCH64, ARM, RISCV64, and *Not sure*. Besides that, a free-form textual field asks how behavior may differ across architectures. This captures qualitative information that cannot be expressed by a fixed dropdown list.

This can help bug triagers and bug fixers quickly determine the affected architectures. Moreover, it helps distinguish architecture-specific failures from general bugs. It may also offer benefits in the software supply chain security and management since companies in the ecosystem can easily decide whether an issue would be relevant to their repository.

Additionally, we propose a new field in the bug reports that indicates the issue category. Issues in the issue tracking systems are commonly known as bug reports although not all of them are software defects (i.e., actual bugs). Some of them are descriptions of new-feature requests. Others may be issues that deserve special treatments. For example, TianoCore treats security vulnerabilities in a special manner and through a separate process different from non-security issues. Furthermore, we noticed that there might be some interest in labeling regression bugs. These are cases in which a software unit would work previously and would now be broken as a result of some development or maintenance activity. Therefore, we propose four possible options for the bug report category that can be in the form of a tag (a report may have several tags simultaneously): (i) New-feature requests, (ii) Functionality-related bugs, (iii) Regression bugs, and (iv) Security-related bugs. Note that security vulnerabilities are often handled through a separate process. The TianoCore community prefers the bug reporters to report such issues in a separate dedicated tab in the GitHub repository, named *Security and quality*. There exists a documented process for responsible disclosure with an embargo period. In the current implementation of our prototype, only the regression category has been addressed. This is shown in Figure 2.

We also assist in bug triage without excluding human triagers in making decisions. This is achieved by advisory comments that are generated by AI-based methods. These comments automatically populate in the bug reports to provide additional information. However, they can be distinguished from the original bug report content. This information should be reviewed by developers (i.e., bug triagers, package maintainers, and bug fixers). We continuously seek feedback from them and loop this back into our AI-based method for improvements. In the present prototype, the AI-generated comments include suggestions from the AI method concerning the following: i) The validity of the bug report (is this really a bug?); ii) Duplicate detection (is there another issue in the repository reporting the same matter?); iii) Priority determination (is this a low/medium/high priority issue?); iv) Bug report assignment (who should fix this bug?). Furthermore, specifying the impacted packages and/or components automatically using our AI-based methods is a work-in-progress. The advisory outputs used in our A/B study were generated by TianoForge, an implemented pipeline contributed by co-author Nazanin Siavash [11].

In addition, we provide a confidence score for the comments and determinations of the AI-based method. This confidence score

is provided prior to the human user’s (triager’s, fixer’s, or maintainer’s) review. However, the review and feedback from the user will help adjust and enhance this feature in the future.

Moreover, our pilot interviews with developers indicated that they preferred explainable AI-based recommendations. Therefore, we offer some evidence and justification for each AI-generated advisory comment. For example, the generated text includes reasons for why the *duplicate* or *invalid* tag was suggested for an issue, which piece of bug report information supports a particular priority level for a bug report, or what historical data in the software repositories signal the fitness of an assignee to resolve the issue.

4 Preliminary Results

In this section, we present some preliminary outcomes of this work-in-progress study.

4.1 Methodology

The key underlying research question is how we can adapt the current bug report structure in TianoCore to make bug triage and resolution more effective and efficient. The motivating factor for this was our observation concerning the very large number of open (and untriaged) issues in the issue tracking system, many of them high-priority bug reports.

To answer the above research question, we first review the information in the TianoCore software repositories, primarily the bug repository. We manually analyze 92 bug reports from the EDK II repository, drawn from a documented snapshot of all closed, type:bug-labeled GitHub issues that were originally opened on GitHub. We call the issues that were originally opened on GitHub Issues, *GitHub-native*. The TianoCore community has migrated from the Bugzilla issue tracking system to GitHub Issues. Therefore, we call the issues that have been transferred from Bugzilla to GitHub Issues non-native to GitHub. The latter category is identifiable by a "Bugzilla Bug #" reference in the issue title. The snapshot was captured on June 30, 2026, via the GitHub GraphQL API. The fetch script is available in the public source artifact [16], and the raw issue data and coded results are archived in Harvard Dataverse [2]. For each issue, we record whether the report describes a regression, whether it mentions CPU architecture information, and how architecture information relates to diagnosis. The corpus analysis is used to motivate the new template fields that can affect bug triage and resolution. Manual analysis was the primary signal for these labels; a separate LLM-assisted analysis was used only as an additional consistency check.

We also plan to conduct interviews, including A/B testing, with developers who engage in bug triage and/or bug fixing in TianoCore. Four pilot interviews have already been conducted to gain insight and prepare for the actual ones. We deploy a within-subjects A/B comparison. Participants review pairs of real EDK II issues in two representations shown under the blinded labels Template 1 and Template 2. The website deterministically randomizes both issue order and the mapping of the baseline and modified representations to these labels using each participant ID. The two issue examples in the current comparison tool are issue 11374, a runtime-driver memory-permissions regression, and issue 11882, a RedfishPkg IA32 NOOPT linker failure. We ask participants to think aloud,

complete the on-screen questions, and answer verbal follow-up open questions about general preference, usefulness of the information, missing or confusing information, structure of the fields, information representation, etc. In the current pilot interviews, the population includes four participants enrolled in Ph.D. studies in a relevant field with one among them being a TianoCore security expert with industry development experience in TianoCore.

4.2 Repository Analysis

Figure 3 illustrates the frequency of regression bugs and the frequency of bug reports with architecture-related information in our selected corpus of 92 bug reports. 11 out of 92 issues were labeled as regressions. Also, 35 out of 92 bug reports mentioned architecture information. Furthermore, 16 issues used architecture information in a way that was trigger-specific or root-cause-specific. This indicates that architecture is not merely background context. Instead, in many cases it changes how the issue should be diagnosed.

4.3 Pilot Interviews

During the interview sessions conducted through video calls over the Internet, we asked the participant to share their screen, open the web-based prototype shown in Figures 4 and 5 in their web browsers, read the provided information there, and answer the questions. Three out of four participants preferred the modified version with additional fields and AI support. However, one participant, who was the more experienced one, was critical of most additions and preferred the more concise version for bug triage. This participant was also critical of the original template and did not believe either of the variants would help them triage and/or resolve issues more effectively or efficiently. We intend to hold more discussions and feedback rounds with this participant, as well as the other TianoCore developers to incorporate their feedback into the prototypes.

5 Conclusion and Future Work

In this paper, we proposed an improved structure for bug report templates of the TianoCore EDK II that makes firmware-specific diagnostic fields explicit and integrates advisory AI comments for bug triagers and fixers. A manual analysis of 92 EDK II issues shows that regression and architecture information appear often enough to justify structured fields, while the exploratory pilot interviews provided formative design feedback on the new structured representation.

This is a work-in-progress. We plan to conduct the main interviews with developers of TianoCore at a larger scale using the demonstrated web-based prototype. In addition, we will validate the quality of the AI-generated advisory comments with developers in various roles, including bug triagers, bug fixers, and package maintainers. Future iterations will place each recommendation and its linked supporting evidence before model-identification details to better support rapid issue scanning.

Software and Data Availability

Our source code is publicly available at [16]; the corresponding replication data are archived in Harvard Dataverse [2].

Which targets are impacted by this bug?

Select all that apply

None ▾

Which architectures are impacted?

Select all architectures where this issue is observed. If unknown, select "Not Sure".

None ▾

Architecture Impact Details

Describe how the issue behaves across architectures (if known). Example: fails on IA32 but works on X64.

Write Preview H B I ≡ <> 🔗 ≡ ≡ ≡ @ ↗ ☑ ↶

Leave a comment

📎 Paste, drop, or click to add files

Current Behavior *

A concise description of the bug that you're experiencing.

Write Preview H B I ≡ <> 🔗 ≡ ≡ ≡ @ ↗ ☑ ↶

Leave a comment

Figure 1: The impacted architectures added to the bug report template

Acknowledgments

This material is based upon work supported by the U.S. National Science Foundation (NSF) under Grant No. 2534021. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the NSF. In preparing this work, we used generative AI models and tools, including OpenAI GPT and Anthropic Claude models, to assist in generating and revising code and text.

Add a title *

[Bug]: <title>

👏 Thanks for taking the time to fill out this bug report!

Is there an existing issue for this?

Please search to see if an issue already exists for the bug you encountered. [Search existing issues](#)

☐ I have searched existing issues *

Did this work before? *

Indicate whether this issue appeared after a change or update.

Not Sure ▾

Bug Type

What type of code does this bug affect?

☐ Firmware

☐ Tool

☐ Unit Test

What packages are impacted? *

Select all that apply

None ▾

Which targets are impacted by this bug?

Figure 2: Identifying regression bugs in the new bug reports template

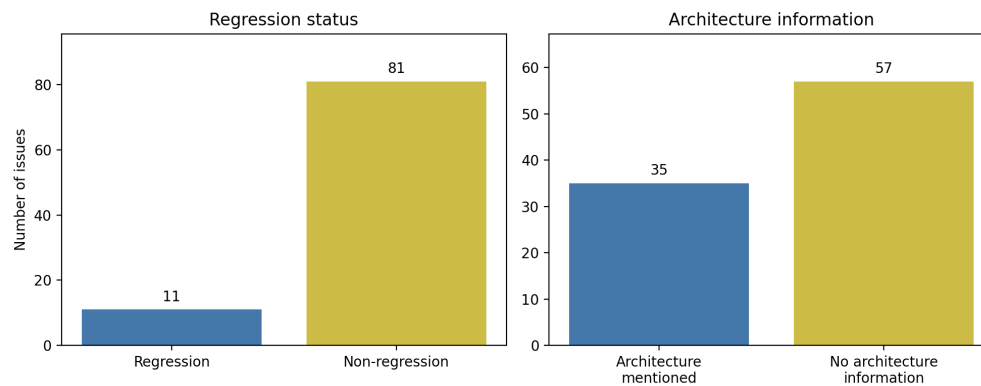


Figure 3: According to our bug repository analysis, regression and architecture information appear frequently enough to justify explicit fields for them in the bug reports.

What is your role? *

Which of these describe how you work with EDK II issues? Please select all that apply.

☐ Bug triager

☐ Bug fixer

☐ Maintainer

☐ Other

Figure 4: Screenshot from our web-based prototype for validation. Here, we are asking about the participant’s primary role.

Runtime driver memory permissions regression: compare the two versions

Issue #11374 · Bug report 1 of 2 · [Next: a different bug report](#)

Please look at both versions side by side and answer below. Please use the toggle to focus on a single template at a time.

☒ Side by side
 ☐ Template 1 only
 ☐ Template 2 only

[Bug]: UEFI runtime drivers loaded after EndOfDxe are not executable by OS #11374

Open · reviewer opened this issue · 1 comment

TEMPLATE 1

Is there an existing issue for this?

- ☒ I have searched existing issues

Bug Type

- ☒ Firmware
- ☐ Tool
- ☐ Unit Test

What packages are impacted?

MdeModulePkg, OvmfPkg

Which targets are impacted by this bug?

DEBUG, NOOPT, RELEASE

Current Behavior

... a runtime Driver after EndOfDxe event and then call it from the operating system

TEMPLATE 2

Is there an existing issue for this?

- ☒ I have searched existing issues

Did this work before?

- ☒ Yes - This is a regression
- ☐ No
- ☐ Not sure

Bug Type

- ☒ Firmware
- ☐ Tool
- ☐ Unit Test

What packages are impacted?

MdeModulePkg, OvmfPkg

Which targets are impacted by this bug?

Figure 5: Screenshot from our web-based prototype for validation. The figure depicts two alternative views (original and enhanced with augmented fields) for the same bug report side by side.

References

- [1] Jagrit Acharya and Gouri Ginde. 2025. Can We Enhance Bug Report Quality Using LLMs?: An Empirical Study of LLM-Based Bug Report Generation. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*. Association for Computing Machinery, New York, NY, USA, 994–1003. doi:10.1145/3756681.3756995
- [2] Laura Baird, Neelesh Reddybattula, Nazanin Siavash, Terrance E. Boulton, and Armin Moin. 2026. Replication Data for Enhancing Bug Report Templates in the TianoCore UEFI Firmware Development Community. Harvard Dataverse. doi:10.7910/DVN/BTZFCQ
- [3] Oscar Chaparro, Carlos Bernal-Cardenas, Jing Lu, Kevin Moran, Andrian Marcus, Massimiliano Di Penta, Denys Poshyvanyk, and Vincent Ng. 2019. Assessing the Quality of the Steps to Reproduce in Bug Reports. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. Association for Computing Machinery, New York, NY, USA, 86–96. doi:10.1145/3338906.3338947
- [4] Oscar Chaparro, Jing Lu, Fiorella Zampetti, Laura Moreno, Massimiliano Di Penta, Andrian Marcus, Gabriele Bavota, and Vincent Ng. 2017. Detecting Missing Information in Bug Descriptions. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*. Association for Computing Machinery, New York, NY, USA, 396–407. doi:10.1145/3106237.3106285
- [5] Morakot Choetkiertikul, Hoa Khanh Dam, Truyen Tran, Trang Pham, Chaiyong Ragkhitwetsagul, and Aditya Ghose. 2021. Automatically Recommending Components for Issue Reports Using Deep Learning. *Empirical Software Engineering* 26, 2 (2021), 14. doi:10.1007/s10664-020-09898-5
- [6] Jianjun He, Ling Xu, Meng Yan, Xin Xia, and Yan Lei. 2020. Duplicate Bug Report Detection Using Dual-Channel Convolutional Neural Networks. In *Proceedings of the 28th International Conference on Program Comprehension*. Association for Computing Machinery, New York, NY, USA, 117–127. doi:10.1145/3387904.3389263
- [7] Hongyan Li, Meng Yan, Weifeng Sun, Xiao Liu, and Yunsong Wu. 2023. A First Look at Bug Report Templates on GitHub. *Journal of Systems and Software* 202 (2023), 111709. doi:10.1016/j.jss.2023.111709
- [8] Zhixing Li, Yue Yu, Tao Wang, Yan Lei, Ying Wang, and Huaimin Wang. 2023. To Follow or Not to Follow: Understanding Issue/Pull-Request Templates on GitHub. *IEEE Transactions on Software Engineering* 49, 4 (2023), 2530–2544. doi:10.1109/TSE.2022.3224053
- [9] Nafiseh Nikeghbal, Amir Hossein Kargaran, and Abbas Heydarnoori. 2024. GIRT-Model: Automated Generation of Issue Report Templates. In *Proceedings of the 21st International Conference on Mining Software Repositories*. Association for Computing Machinery, New York, NY, USA, 407–418. doi:10.1145/3643991.3644906
- [10] Cheng Qian, Ming Zhang, Yuanping Nie, Shuaibing Lu, and Huayang Cao. 2023. A Survey on Bug Deduplication and Triage Methods from Multiple Points of View. *Applied Sciences* 13, 15 (2023), 8788. doi:10.3390/app13158788
- [11] Nazanin Siavash, Terrance E. Boulton, and Armin Moin. 2026. TianoForge: Automated Bug Triage for TianoCore EDK II. <https://github.com/TianoShield/TianoForge>.
- [12] Mozhan Soltani, Felienne Hermans, and Thomas Back. 2020. The Significance of Bug Report Elements. *Empirical Software Engineering* 25, 6 (2020), 5255–5294. doi:10.1007/s10664-020-09882-z
- [13] Yang Song, Junayed Mahmud, Ying Zhou, Oscar Chaparro, Kevin Moran, Andrian Marcus, and Denys Poshyvanyk. 2022. Toward Interactive Bug Reporting for (Android App) End-Users. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. Association for Computing Machinery, New York, NY, USA, 344–356. doi:10.1145/3540250.3549131
- [14] TianoCore Project. 2026. TianoCore Bug Triage. https://www.tianocore.org/tianocore-wiki.github.io/development/contribution-guides/bug_triage.html. Accessed 2026-08-17.
- [15] TianoCore Project. 2026. TianoCore EDK II GitHub issue templates. https://github.com/tianocore/edk2/tree/master/.github/ISSUE_TEMPLATE. Accessed 2026-08-17.
- [16] TianoShield Project. 2026. TianoCore Bug Report Templates: Source Code and Reproducibility Artifact. <https://github.com/TianoShield/tianocore-bug-report-templates>.
- [17] Junwei Zhang, Zhongxin Liu, Lingfeng Bao, Zhenchang Xing, Xing Hu, and Xin Xia. 2024. Inside Bug Report Templates: An Empirical Study on Bug Report Templates in Open-Source Software. In *Proceedings of the 15th Asia-Pacific Symposium on Internetware*. Association for Computing Machinery, New York, NY, USA, 125–134. doi:10.1145/3671016.3671401
- [18] Thomas Zimmermann, Rahul Premraj, Nicolas Bettenburg, Sascha Just, Adrian Schroter, and Cathrin Weiss. 2010. What Makes a Good Bug Report? *IEEE Transactions on Software Engineering* 36, 5 (2010), 618–643. doi:10.1109/TSE.2010.63